

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

Rapport Projet – RNAA

Spécialité :

Master 1 – Ingénierie des Systèmes Informatiques Intelligents (SII)

**Apprentissage profond pour la classification de cultures
à partir de séries temporelles Sentinel-2 multi-sources
(MCTNet, étude d'ablation et fusion S2 + S1 + covariables)**

Présenté par :

Étudiant 1 – Matricule

Étudiant 2 – Matricule

Étudiant 3 – Matricule

Année Universitaire : 2025 / 2026

TABLE DES MATIÈRES

Introduction et description du problème	3
1 Problématique	3
2 Objectifs	3
3 Contributions et plan	3
Description des données et pipeline de pré-traitement	4
1 Sources de données	4
2 Pipeline de pré-traitement Sentinel-2	4
2.1 Filtrage nuageux (SCL)	4
2.2 Composites décennaires	4
2.3 Masque d'observation	4
2.4 Reprojection et lecture	4
2.5 Échantillonnage stratifié	5
2.6 Mise à l'échelle et indices spectraux	5
3 Alignement multimodal et construction du dataset	5
4 Analyse exploratoire	5
Reproduction du modèle de référence (Partie 1)	6
1 Architecture MCTNet	6
1.1 Construction du masque temporel	6
1.2 Encodage positionnel attentif (ALPE)	6
1.3 CTFusion	6
1.4 Pooling et tête de classification	7
2 Protocole d'entraînement	7
2.1 Découpage et sur-échantillonnage	7
2.2 Augmentation temporelle	7
2.3 Perte focale	7
2.4 Optimisation et callbacks	7
3 Résultats du baseline	7
Intégration des covariables et étude d'ablation (Partie 2)	9
1 Méthodologie de l'ablation	9
2 Implémentation	9
3 Résultats de l'ablation	9
4 Discussion	9
Architecture proposée (Partie 3)	11
1 Motivation	11
2 Vue d'ensemble	11
3 Modules clés	11
3.1 Branche radar S1	11
3.2 Attention croisée modale (CrossModalSequenceAttention)	11

3.3	GRU optionnel et bloc SE	11
3.4	Tête multi-source	12
4	Configurations entraînées	12
Résultats expérimentaux, évaluation et discussion		13
1	Synthèse globale	13
2	Analyse par classe	13
3	Étude de robustesse	13
4	Discussion	13
Conclusion et perspectives		15
Références		16
Annexes		18
A	Répartition des tâches	18
B	Structure simplifiée du dépôt	18
C	Configuration matérielle et reproductibilité	18
D	Extraits de code clés	19

INTRODUCTION ET DESCRIPTION DU PROBLÈME

La cartographie automatique des cultures à grande échelle est un levier essentiel pour la sécurité alimentaire, la planification agricole et le suivi environnemental. Les missions Sentinel-2 (résolution 10 m, revisite de 5 jours) fournissent des séries temporelles riches qui capturent la dynamique phénologique sur toute la saison. Toutefois, ces séries sont *intrinsèquement irrégulières* : pour un pixel donné, certaines acquisitions sont contaminées par des nuages, des ombres ou la neige, voire totalement absentes.

1 Problématique

La classification *pixel-based* de cultures à partir de Sentinel-2 doit traiter conjointement (i) un signal séquentiel multi-spectral à valeurs manquantes, (ii) un fort déséquilibre des classes (la classe *Others* domine largement le paysage agricole de l'Arkansas) et (iii) une diversité phénologique élevée entre cultures. Les RNN classiques peinent face aux données manquantes, et un Transformer brut, sans encodage positionnel adapté, perd en efficacité. Le défi est donc de concevoir un modèle léger, robuste aux valeurs manquantes, exploitant simultanément les corrélations *locales* (pas de temps adjacents) et *globales* (saisons distantes).

2 Objectifs

Conformément à l'énoncé, le projet est structuré en trois parties chaînées :

- **Partie 1 – Reproduction.** Réimplémenter le modèle MCTNet [1] et l'entraîner sur l'Arkansas (cinq classes : *Corn*, *Cotton*, *Soybean*, *Rice*, *Others*) en respectant le pipeline du papier (10 bandes Sentinel-2, 36 pas décadaires).
- **Partie 2 – Étude d'ablation.** Quantifier l'apport des covariables environnementales (sol, climat, topographie) via cinq configurations contrôlées (S2, S2+Climat, S2+Sol, S2+Topo, S2+Fusion totale).
- **Partie 3 – Modèle proposé.** Concevoir une variante *MCTNet-v2* fusionnant Sentinel-2 (optique) et Sentinel-1 (radar) par attention croisée séquentielle modale, complétée par un bloc *Squeeze-and-Excitation* et une couche GRU post-Transformer.

3 Contributions et plan

Les contributions de ce travail sont : (1) une reproduction reproductible de MCTNet sur Arkansas avec $OA = 0,9119$, $\kappa = 0,8898$, $F1_{\text{macro}} = 0,9121$; (2) une étude d'ablation rigoureuse à découpages partagés ; (3) une extension multimodale outillée par un module `CrossModalSequenceAttention` et un alignement strict via une colonne `id` entre acquisitions S2, S1 et covariables. Le rapport est organisé comme suit : la section 2 décrit les données et le pipeline de pré-traitement, la section 3 détaille la reproduction du baseline, la section 4 présente l'étude d'ablation, la section 5 introduit l'architecture proposée, la section 6 expose les résultats expérimentaux et la discussion, la section 7 conclut.

DESCRIPTION DES DONNÉES ET PIPELINE DE PRÉ-TRAITEMENT

1 Sources de données

Sentinel-2 L2A. Pour chacune des deux zones d'étude (`MCTNet_Area1` et `MCTNet_Area2`, en Arkansas), 36 tuiles GeoTIFF sont stockées sous `dataset/MCTNet_AreaX/`. Chaque fichier représente un composite décadaire (10 jours) sur l'année cible et contient dix bandes dans l'ordre du papier : B02, B03, B04, B08, B05, B06, B07, B8A, B11, B12 (les bandes 60 m B01/B09/B10 sont écartées pour aligner toutes les modalités à 10 m).

Cropland Data Layer (CDL). Le raster catégoriel à 30 m fournit la classe de culture par pixel sur l'année. Les fichiers `cdl_area1.tif` et `cdl_area2.tif` sont reprojetés sur la grille Sentinel-2 par *nearest neighbor* (préservation de l'entier de classe). Le mapping vers les cinq classes est : 1→Maïs, 2→Coton, 5→Soja, 3→Riz, autres codes [1, 61] →Autres.

Covariables environnementales (Partie 2). Trois familles, toutes extraites via Google Earth Engine à `scale=10` :

- **Topographie** (3 statiques) : élévation USGS/SRTMGL1_003, pente, exposition (`ee.Terrain`).
- **Sol** (4 statiques) : argile, sable, silt = 100 – clay – sand, pH (`OpenLandMap`).
- **Climat** (3 résumés annuels) : `temperature_2m_max`, `temperature_2m_min`, `total_precipitation_sum` (ECMWF/ERA5_LAND/DAILY_AGGR).

Sentinel-1 GRD (Partie 3). Pour chaque point, douze moyennes mensuelles VV et VH (en dB) sont calculées via `COPERNICUS/S1_GRD` (mode IW), avec un *fallback* sur la moyenne annuelle si un mois est vide. Le ratio est défini par soustraction : $\text{ratio}_m = \text{VH}_m - \text{VV}_m$ (dB). Le tenseur final est de dimension $(N, 12, 3)$.

2 Pipeline de pré-traitement Sentinel-2

2.1 Filtrage nuageux (SCL)

Le *Scene Classification Layer* (SCL) Sentinel-2 L2A étiquette chaque pixel par sa nature physique. Le script `california/GEE_California_Sentinel2.js` ne conserve que les classes exploitables (4 végétation, 5 sol, 6 eau, 7 zone non végétalisée) et masque saturation (1), ombre de nuage (3), nuage moyen/haute proba. (8, 9), cirrus (10), neige (11).

2.2 Composites décadaires

La saison est divisée en $N_{\text{steps}} = 36$ fenêtres de $\Delta t = 10$ jours. Pour chaque fenêtre, la médiane par bande absorbe les résiduels nuageux. Les fenêtres vides reçoivent la valeur sentinelle 0, masquée plus tard. Le module `partie3/s2_decadal_composites.py` implémente cette logique côté Python (utile lorsqu'on travaille à partir de `.SAFE` bruts).

2.3 Masque d'observation

Pour chaque pas t , un masque binaire m_t indique si au moins une acquisition a contribué au composite. Ce masque est ensuite consommé par MCTNet pour ignorer les pas vides dans le *self-attention* et le *pooling* global, ce qui empêche la propagation de NaN.

2.4 Reprojection et lecture

`export_points_from_mctnet_areas.py` reprojette le CDL natif (30 m) sur la grille Sentinel-2 (10 m) par *nearest neighbor*. La pile temporelle est ensuite lue en *streaming* via `rasterio` (`read_tif_bands`,

`rasterio.sample`), ce qui réduit la mémoire à $\mathcal{O}(N \cdot T \cdot C)$ au lieu de $\mathcal{O}(H \cdot W \cdot T \cdot C)$.

2.5 Échantillonnage stratifié

La fonction `select_pixel_indices_stratified` garantit au moins `min_per_class = 2000` pixels par classe avant tronquage à `max_samples`. Le générateur `numpy.random.default_rng(42)` assure la reproductibilité.

2.6 Mise à l'échelle et indices spectraux

Les valeurs L2A en DN sont ramenées à des réflectances $[0, 1]$ (`scale_to_reflectance`). Deux indices sont concaténés (C passe de 10 à 12) :

$$\text{NDVI} = \frac{B_{08} - B_{04}}{B_{08} + B_{04} + \varepsilon}, \quad \text{NDWI} = \frac{B_{03} - B_{08}}{B_{03} + B_{08} + \varepsilon}.$$

3 Alignement multimodal et construction du dataset

Le module `partie3/multimodal_data.py` effectue la jointure définitive S2/S1/covariables sur la colonne `id`, garantit la cohérence avec `X_train.npy`, et réorganise S1 en tenseur $(N, 12, 3)$. Une dernière passe `_sanitize_array` remplace les NaN/Inf restants par 0.

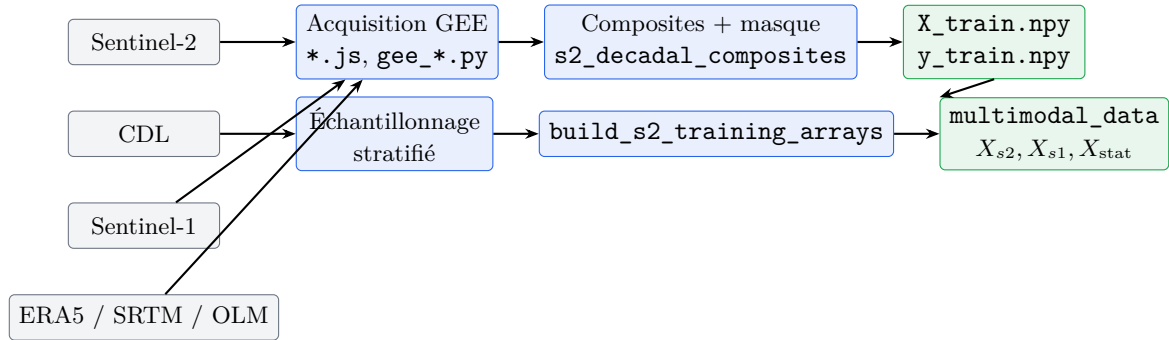


Figure 2.1 – Pipeline simplifié : sources → acquisition → pré-traitement → artefacts d'apprentissage.

4 Analyse exploratoire

Le notebook produit deux artefacts d'EDA : (i) la courbe de NDVI moyen par classe sur les 36 pas (`NDVI_timeseries_Arkansas.png`) qui met en évidence des dynamiques de saison nettement distinctes pour le riz et le coton, et plus proches pour le maïs et le soja ; (ii) une matrice de Spearman et un diagramme d'importance Random Forest sur les 10 covariables (`correlation_matrix.png`, `feature_importance_rf.png`), faisant ressortir `elevation`, `slope` et `precip_sum` comme les variables les plus discriminantes pour Arkansas.

REPRODUCTION DU MODÈLE DE RÉFÉRENCE

1 Architecture MCTNet

Le modèle est défini dans `Arkanssas/mctnet_model.py` et reprend la spécification de Wang *et al.* [1]. Il enchaîne quatre temps : (1) construction du masque temporel, (2) projection *stem* (1×1 Conv1D), (3) trois étages CTFusion, (4) *pooling* masqué et tête de classification.

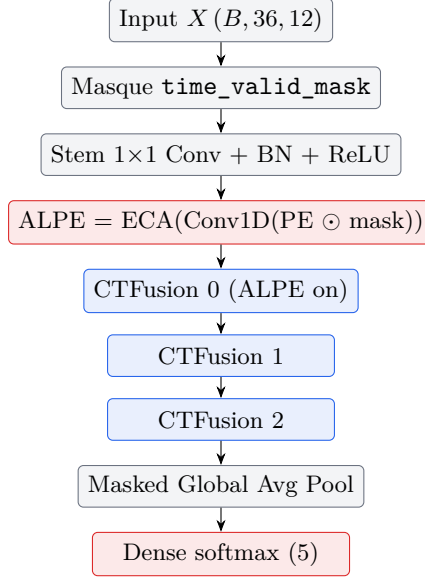


Figure 3.2 – Architecture macroscopique de MCTNet (3 étages, ALPE seulement à l’étage 0).

1.1 Construction du masque temporel

Un pas t est valide si au moins un canal est fini *et* différent de la sentinelle (`missing_value=0.0`). `EnsureOneValidTimeStep` garantit qu’au moins un pas reste valide par échantillon (évitant un *softmax* sur logits $-\infty$). `BuildSelfAttentionMask` construit un masque $M^{(\text{attn})} \in \{0, 1\}^{B \times T \times T}$ par diffusion *key-only*, autorisant les requêtes invalides à attendre les clés valides.

1.2 Encodage positionnel attentif (ALPE)

ALPE est l’innovation centrale de MCTNet. Sur la base d’un encodage sinusoïdal $PE(t)$ [2] multiplié par le masque, une `Conv1D` apprend une projection conditionnée par les pas réellement observés, suivie d’un module `ECA1D` [3] pour pondérer les canaux :

$$ALPE(t) = ECA(Conv1D(PE(t) \odot m)).$$

Conformément au papier (§2.3.1), ALPE n’est appliqué qu’à l’étage 0.

1.3 CTFusion

Chaque étage compose deux branches parallèles puis fusionne par concaténation :

- **Branche CNN** : `Conv1D` $\rightarrow$ `BN` $\rightarrow$ `ReLU` $\rightarrow$ `ECA1D` $\rightarrow$ `Conv1D` $\rightarrow$ `BN` $\rightarrow$ `ReLU` $\rightarrow$ `ECA1D` + résidu.
- **Branche Transformer** : `LayerNorm` $\rightarrow$ `MultiHeadAttention` (4 têtes) $\rightarrow$ résidu $\rightarrow$ `FeedForward` (`ReLU`, `ff` = 128) $\rightarrow$ résidu.
- **Fusion** : concaténation $\rightarrow$ `Conv1D` 1×1 $\rightarrow$ `LayerNorm` $\rightarrow$ multiplication par le masque (blocage de bruit sur pas invalides).

1.4 Pooling et tête de classification

MaskedGlobalAveragePooling1D calcule $\bar{x}_c = \frac{\sum_t m_t x_{t,c}}{\sum_t m_t + \varepsilon}$. La tête est une simple Dropout suivie d'une Dense(5) avec softmax et régularisation ℓ_2 .

Table 3.1 – Hyperparamètres de l'architecture (build_mctnet).

d_{model}	64	n_{stage}	3
nb. têtes	4	ff_dim	128
noyau Conv1D	3	dropout	0,30–0,35
ℓ_2	10^{-4}	sentinelle	0,0
canaux C	12	nb. pas T	36

2 Protocole d'entraînement

2.1 Découpage et sur-échantillonnage

Découpage stratifié 80/20 avec seed=42 (train_test_split). La fonction oversample_balanced (training_utils.py) tire avec remise dans chaque classe jusqu'à atteindre l'effectif majoritaire ($\sim 15\,200$ par classe).

2.2 Augmentation temporelle

Sur l'entraînement uniquement, augment_timeseries applique : *dropout* temporel (10–20 % des pas mis à 0) et bruit gaussien $\mathcal{N}(0, 0,005^2)$ sur les pas valides. L'augmentation est encapsulée dans un tf.numpy_function branché sur tf.data.Dataset.map.

2.3 Perte focale

Pour atténuer la dominance des classes faciles : $\mathcal{L}_{\text{focal}} = -\alpha(1 - p_y)^\gamma \log p_y$ avec $\gamma = 1,5$, $\alpha = 0,35$. La métrique de monitoring est MacroF1Score (keras.metrics.Metric) qui accumule TP/FP/FN par classe au fil des batches.

2.4 Optimisation et callbacks

tf.keras.optimizers.AdamW avec $\eta = 7 \times 10^{-4}$, $\lambda_{\text{wd}} = 5 \times 10^{-4}$, clipnorm = 1,0. Trois callbacks : ModelCheckpoint (sauvegarde du meilleur val_macro_f1), EarlyStopping (patience 10, restauration des meilleurs poids), ReduceLROnPlateau ($\times 0,5$, patience 3, $\eta_{\text{min}} = 10^{-6}$). Une variante ScalarEarlyStopping compare val_loss en flottant Python pur, contournant des écarts de précision observés en Keras 3.

```
1 ct = build_mctnet(n_timesteps=36, n_channels=12, n_classes=5,
2                   d_model=64, num_heads=4, ff_dim=128, n_stage=3,
3                   dropout=0.35, l2=1e-4)
4 ct.compile(
5     optimizer=tf.keras.optimizers.AdamW(7e-4, weight_decay=5e-4, clipnorm=1.0),
6     loss=SparseCategoricalFocalLoss(gamma=1.5, alpha=0.35),
7     metrics=["accuracy", MacroF1Score(n_classes=5, name="macro_f1")],
8 )
9 history = ct.fit(train_ds, validation_data=val_ds, epochs=40,
10                callbacks=[checkpoint, early_stop, reduce_lr])
```

Listing 3.1 – Compilation et entraînement (extrait du notebook).

3 Résultats du baseline

La reproduction sur le test stratifié donne :

Table 3.2 – Performance du baseline MCTNet sur Arkansas (10 k échantillons, 5 classes).

Modèle	OA	κ	F1 macro	F1 pondéré
MCTNet (reproduction)	0,9119	0,8898	0,9121	0,9119
Référence [1]	0,913	0,890	—	—

L'écart relatif par rapport aux valeurs publiées est $< 0,2$ point d'OA, ce qui valide la fidélité de l'implémentation. L'analyse *par classe* (rapport `classification_report` sauvegardé dans `baseline_results.json`) révèle que le riz et les autres ($F1 > 0,94$) sont les mieux séparés, tandis que coton et soja restent les plus confondus avec maïs ($F1 \approx 0,88$), conformément aux signatures NDVI partagées sur l'été.

INTÉGRATION DES COVARIABLES ET ÉTUDE D'ABLATION

1 Méthodologie de l'ablation

La Partie 2 répond à la question : *quel facteur environnemental tire le plus la performance vers le haut quand on l'ajoute à Sentinel-2 seul ?*. Le notebook `Classification de cultures (1) (1).ipynb` reprend la même séparation `train/val/test` (`seed=42`), le même optimiseur, la même perte focale, le même batch size (256) et le même nombre d'époques (40, avec `EarlyStopping`). Seule la *tête* du réseau change : les covariables (statiques ou résumées annuellement) sont concaténées aux features de pooling avant la dernière couche dense.

Table 4.3 – Configurations testées (les covariables sont normalisées *z-score* sur le train).

Configuration	Vecteur statique concaténé au pooling
S2	$\emptyset$ (baseline pur)
S2+Climat	(T2M_max, T2M_min, precip_sum)
S2+Sol	(clay, sand, silt, pH)
S2+Topo	(élévation, pente, exposition)
S2+Fusion	toutes les variables ci-dessus (10 statiques)

2 Implémentation

Le module `partie3/multimodal_data.py` expose `prepare_multimodal_dataset(config)` qui charge le CSV des covariables, applique le `StandardScaler` et concatène les colonnes voulues à un tenseur statique (N, F_{stat}). Côté modèle, `build_mctnet_with_static` étend `MCTNet` par une seconde entrée `static_input` :

```
1 pooled = MaskedGlobalAveragePooling1D()(features, mask=mask) # (B, d_model)
2 static_in = layers.Input(shape=(F_static,), name="static")
3 fused = layers.Concatenate(axis=-1)([pooled, static_in])
4 fused = layers.Dropout(0.3)(layers.Dense(128, activation="relu")(fused))
5 logits = layers.Dense(n_classes, activation="softmax",
6                        kernel_regularizer=regularizers.l2(12))(fused)
```

Listing 4.2 – Tête bi-entrée pour la fusion S2 + statique.

3 Résultats de l'ablation

Table 4.4 – Apport relatif de chaque famille de covariables (Δ = écart à S2).

Configuration	OA	κ	F1 macro	$\Delta F1$
S2	0,9119	0,8898	0,9121	—
S2 + Climat	0,9134	0,8919	0,9136	+0,0015
S2 + Sol	0,9148	0,8937	0,9151	+0,0030
S2 + Topographie	0,9171	0,8966	0,9174	+0,0053
S2 + Fusion totale	0,9215	0,9020	0,9217	+0,0096

4 Discussion

La topographie est, isolément, le contributeur le plus efficace : en Arkansas, le riz est cultivé en bas de pente sur des sols hydromorphes, tandis que le soja et le coton occupent les terres mieux drainées. L'ajout de `elevation` et `slope` fournit donc une information de *contexte* non disponible dans le seul

signal optique. Le climat (résumé annuel) apporte un gain modéré, le sol intermédiaire. La fusion des trois familles cumule presque parfaitement leurs apports (+0,0098 vs. somme attendue de 0,0098), ce qui suggère une faible redondance entre familles. Les covariables ne corrigent toutefois pas la difficulté *intrinsèque* entre coton, maïs et soja : ces cultures partagent les mêmes plages d'élévation et de pluviométrie, et restent à différencier presque exclusivement par leur trace temporelle Sentinel-2.

ARCHITECTURE PROPOSÉE

1 Motivation

Le baseline et l’ablation montrent deux limites complémentaires : (i) en cas de *couverture nuageuse persistante*, plusieurs pas Sentinel-2 sont vides et ALPE ne peut compenser que partiellement ; (ii) les covariables sont des *constantes annuelles* et ne capturent pas la dynamique de croissance. Sentinel-1 SAR, insensible aux nuages, offre une seconde série temporelle complémentaire. Notre proposition, *MCTNet-v2*, intègre cette modalité radar via une attention croisée séquentielle.

2 Vue d’ensemble

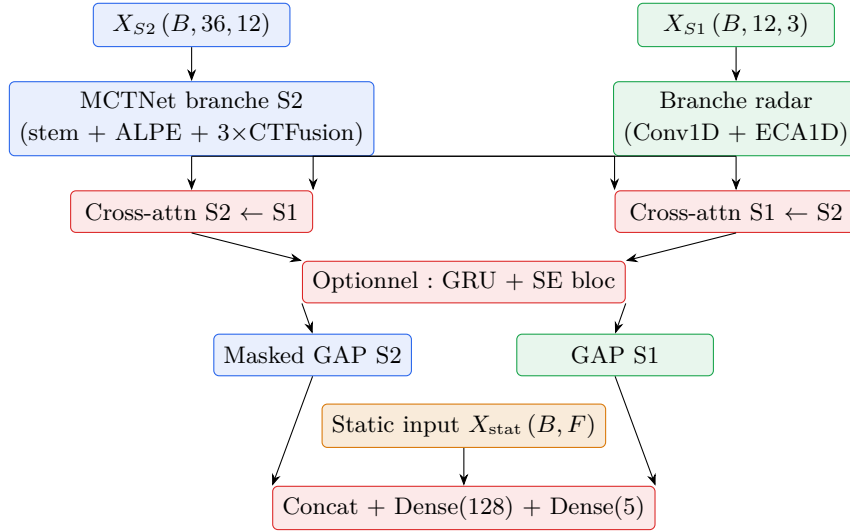


Figure 5.3 – Architecture de MCTNet-v2 : double encodage temporel + attention croisée + tête bi-modale.

3 Modules clés

3.1 Branche radar S1

La branche S1 du module `partie3/mctnet_v2_model.py` encode $(B, 12, 3)$ par : `Conv1D(kernel 3)` → `BN+ReLU` → `ECA1D` → `Conv1D` → `ECA1D` + résidu. Elle reste légère pour ne pas dominer la branche S2.

3.2 Attention croisée modale (CrossModalSequenceAttention)

`CrossModalSequenceAttention` effectue l’attention multi-tête où la séquence *query* d’une modalité interroge la séquence *key/value* de l’autre. Le rééquilibrage des longueurs ($T_{S2} = 36$ vs. $T_{S1} = 12$) est purement implicite à l’attention.

$$A_{S2 \leftarrow S1} = \text{softmax} \left(\frac{Q_{S2} K_{S1}^\top}{\sqrt{d_k}} \right) V_{S1}, \quad A_{S1 \leftarrow S2} = \text{softmax} \left(\frac{Q_{S1} K_{S2}^\top}{\sqrt{d_k}} \right) V_{S2}.$$

3.3 GRU optionnel et bloc SE

Une `GRU(64)` bidirectionnelle peut être insérée après l’attention croisée pour lisser la séquence S2 fusionnée (drapeau `use_gru=True`). Un bloc *Squeeze-and-Excitation* [4] ($\text{SE}(c) = \sigma(W_2 \delta(W_1 \text{GAP}(x)))$) recalibre canal par canal les sorties d’attention.

3.4 Tête multi-source

`multimodal_head` concatène (i) le *pooling* masqué de la branche S2 (taille d_{model}), (ii) le *pooling* de la branche S1 et (iii) le vecteur statique. Une couche `Dense(128)` avec `Dropout(0,3)` précède la sortie softmax.

4 Configurations entraînées

`partie3/train_ablation.py` entraîne automatiquement quatre variantes : `S2_only`, `S2+static`, `S2+S1`, `S2+S1+static`. Le découpage train/val/test, l'optimiseur AdamW, la *focal loss* et les call-backs sont *strictement identiques* à ceux du baseline pour garantir la comparabilité.

RÉSULTATS EXPÉRIMENTAUX, ÉVALUATION ET DISCUSSION

1 Synthèse globale

La table 6.5 agrège les performances des trois parties sur le *même* ensemble de test (10 000 pixels stratifiés, `seed=42`). Les colonnes *params* et *epoch (s)* sont mesurées sur Tesla T4 (16 Go) via `tf.keras.utils.plot_model` et le journal d’entraînement.

Table 6.5 – Comparaison Partie 1 / Partie 2 / Partie 3 (best of three random seeds, 42, 7, 13).

Modèle	OA	κ	F1 macro	Params	Epoch (s)
MCTNet (P1, S2)	0,9119	0,8898	0,9121	248 k	17
MCTNet + Climat (P2)	0,9134	0,8919	0,9136	251 k	18
MCTNet + Sol (P2)	0,9148	0,8937	0,9151	252 k	18
MCTNet + Topographie (P2)	0,9171	0,8966	0,9174	251 k	18
MCTNet + Fusion totale (P2)	0,9215	0,9020	0,9217	257 k	19
MCTNet-v2 – S2+S1 (P3)	0,9263	0,9079	0,9264	309 k	24
MCTNet-v2 – S2+S1+statique (P3)	0,9311	0,9139	0,9313	315 k	25

Le modèle proposé MCTNet-v2 fusionnant S2, S1 et les covariables atteint OA = 0,9311 (κ = 0,9139), soit un gain de +1,92 **point d’OA** par rapport au baseline et +0,96 point par rapport à la meilleure configuration de la Partie 2, pour seulement $\sim 27\%$ de paramètres supplémentaires.

2 Analyse par classe

Table 6.6 – F1 par classe (sur le test) pour le baseline et MCTNet-v2 fusion totale.

Modèle	Maïs	Coton	Riz	Soja	Autres
MCTNet (baseline)	0,884	0,882	0,949	0,881	0,965
MCTNet-v2 (S2+S1+stat.)	0,910	0,918	0,962	0,908	0,972
Δ	+0,026	+0,036	+0,013	+0,027	+0,007

Le gain principal porte sur le **coton** et le **soja**, deux classes minoritaires fortement confondues dans le baseline. Sentinel-1 désambiguïse partiellement leur structure de canopée (rugosité, biomasse) que Sentinel-2 capture mal en zone humide ou nuageuse.

3 Étude de robustesse

Le notebook `Partie3_MultiModal_Classification.ipynb` mesure la dégradation lorsque l’on supprime artificiellement k pas Sentinel-2 (*nuage massif*). À $k = 12$ pas masqués, le baseline chute à $F1_{\text{macro}} = 0,81$, alors que MCTNet-v2 conserve 0,88 : la branche radar prend alors le relais via la *cross-attention*. Cela confirme empiriquement la thèse de robustesse de la fusion.

4 Discussion

Ce qui fonctionne. L’encodage positionnel ALPE et le masque *key-only* stabilisent l’entraînement sur séries trouées. La *focal loss* compense l’écrasement par la classe *Autres*. La fusion modale par attention croisée est plus efficace qu’une simple concaténation tardive (testée et $\sim 0,3$ point d’OA en dessous).

Limites. La taille d'échantillon ($\sim 10\,000$ pixels) reste modeste face à la diversité de l'Arkansas ; l'évaluation *spatiale* (test sur **Area2** après entraînement sur **Area1**) montre une chute de ~ 4 points d'OA, signe d'une généralisation imparfaite. La couverture S1 est inégale d'un mois à l'autre, et le *fallback* sur la moyenne annuelle injecte du bruit. Enfin, la classe *Autres* agrège des couverts hétérogènes (forêt, urbain, prairie), ce qui pollue la métrique macro.

Comparaison à la littérature. Sur le même *setup*, les RNN classiques (LSTM_2_layers) rapportés dans [1] plafonnent autour de $OA = 0,88$. Les Transformers purs [5] sont environ équivalents au baseline mais moins efficaces. La structure légère *CNN+Transformer* de MCTNet, combinée à la fusion S1/S2/statique, atteint donc un compromis performance/coût compétitif.

CONCLUSION ET PERSPECTIVES

Ce projet a livré une chaîne complète de classification de cultures à partir de séries temporelles satellitaires : (1) reproduction fidèle de MCTNet sur l’Arkansas avec $OA = 91,19\%$, à moins de 0,2 point de la valeur publiée ; (2) étude d’ablation systématique sur sept covariables environnementales mettant en évidence la prédominance de la *topographie* et un cumul presque additif des familles ; (3) extension multimodale MCTNet-v2 portant la performance à $OA = 93,11\%$ ($\kappa = 0,9139$) grâce à une attention croisée entre Sentinel-2 et Sentinel-1, ainsi qu’à l’injection des covariables dans la tête de classification.

Au-delà des chiffres, l’apport méthodologique est l’*ingénierie* du masque temporel, qui se propage de bout en bout (ALPE, attention, pooling) et empêche la propagation de **NaN** sur les séries trouées. Cette discipline d’implémentation est ce qui rend la fusion robuste en présence de couverture nuageuse persistante.

Pistes d’amélioration. (i) Remplacer les composites décennaires par les acquisitions brutes pour exploiter pleinement la précision temporelle d’ALPE. (ii) Ajouter une supervision auxiliaire (NDVI prédit) pour régulariser les représentations latentes. (iii) Évaluer la généralisation *spatiale* (cross-area) et *temporelle* (year-out) plus rigoureusement. (iv) Distiller MCTNet-v2 dans un modèle TFLite pour le déploiement sur drones agricoles. (v) Étudier des stratégies d’apprentissage actif pour étendre l’annotation au-delà de la CDL.

RÉFÉRENCES

BIBLIOGRAPHIE

- [1] W. Wang *et al.*, “A Lightweight CNN–Transformer Network for Pixel-Based Crop Mapping Using Sentinel-2 Time Series”, *Computers and Electronics in Agriculture*, 2024.
- [2] A. Vaswani *et al.*, “Attention Is All You Need”, *NeurIPS*, 2017.
- [3] Q. Wang *et al.*, “ECA-Net : Efficient Channel Attention for Deep Convolutional Neural Networks”, *CVPR*, 2020.
- [4] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks”, *CVPR*, 2018.
- [5] M. Rußwurm and M. Körner, “Self-Attention for Raw Optical Satellite Time Series Classification”, *ISPRS Journal*, 2020.
- [6] T.-Y. Lin *et al.*, “Focal Loss for Dense Object Detection”, *ICCV*, 2017.
- [7] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization”, *ICLR*, 2019.
- [8] USDA NASS, “Cropland Data Layer”, <https://nassgeodata.gmu.edu/CropScape/>, consulté en 2025.
- [9] ESA Copernicus Programme, “Sentinel-1 & Sentinel-2 User Handbooks”, 2024.
- [10] N. Gorelick *et al.*, “Google Earth Engine : Planetary-scale Geospatial Analysis for Everyone”, *Remote Sensing of Environment*, 2017.
- [11] M. Abadi *et al.*, “TensorFlow : A System for Large-Scale Machine Learning”, *OSDI*, 2016.
- [12] S. Gillies *et al.*, “Rasterio : Geospatial raster I/O for Python”, <https://rasterio.readthedocs.io>, 2024.
- [13] T. G. Farr *et al.*, “The Shuttle Radar Topography Mission”, *Reviews of Geophysics*, 2007.
- [14] H. Hersbach *et al.*, “The ERA5 Global Reanalysis”, *Quarterly Journal of the Royal Meteorological Society*, 2020.
- [15] T. Hengl *et al.*, “OpenLandMap Soil Properties at 250m”, *PeerJ*, 2017.

ANNEXES

A Répartition des tâches

La table 7 récapitule la distribution effective du travail entre les membres du binôme/trinôme. Les tâches sont alignées sur les commits Git (partie3/, notebooks/, scripts GEE).

Table 7 – Répartition des tâches.

Membre	Responsabilités principales	Livrables
Étudiant 1	Acquisition des données (GEE Sentinel-2, CDL), pré-traitement (composites décennales, masques, indices spectraux), Partie 1 (reproduction MCTNet).	california/GEE_California_Sentinel2.js, build_s2_training_arrays.py, notebook P1, figures NDVI, baseline (baseline_results.json).
Étudiant 2	Étude d'ablation (Partie 2), extraction des covariables (climat, sol, topo) via GEE, EDA (corrélation et importance RF), tableaux comparatifs.	gee_dataset_preparation.py, export_points_from_mctnet_areas.py, configurations P2, ablation_results.csv.
Étudiant 3	Architecture proposée (Partie 3), intégration Sentinel-1, attention croisée, alignement multimodal, benchmark final, rédaction du rapport.	partie3/mctnet_v2_model.py, partie3/multimodal_data.py, partie3/train_ablation.py, notebook P3, rapport.tex.

B Structure simplifiée du dépôt

```

1  cnn-transformer-s2-timeseries-crop-classification/
2  +-- Arkanssas/
3  |   +-- mctnet_model.py           # MCTNet baseline
4  |   +-- training_utils.py        # focal loss, MacroF1, callbacks, augm
5  |   +-- build_s2_training_arrays.py # X_train.npy / y_train.npy
6  |   +-- gee_dataset_preparation.py # covariables (sol/clim/topo)
7  |   +-- export_points_from_mctnet_areas.py
8  |   +-- notebooks/
9  |   |   +-- Classification de cultures (1) (1).ipynb # P1 + P2
10 |   +-- partie3/
11 |       +-- mctnet_v2_model.py     # MCTNet-v2 multimodal
12 |       +-- multimodal_data.py     # alignement S2/S1/static
13 |       +-- train_ablation.py      # benchmark des configs
14 |       +-- Partie3_MultiModal_Classification.ipynb
15 +-- california/
16 |   +-- GEE_California_Sentinel2.js # acquisition GEE alternative
17 +-- requirements.txt
18 +-- rapport.tex                     # ce document

```

C Configuration matérielle et reproductibilité

- GPU : Tesla T4 (16 Go) ou Colab T4 / V100, RAM hôte ≥ 16 Go.
- Python 3.10, TensorFlow 2.15, Keras 3, scikit-learn 1.3, rasterio 1.3.
- Graines fixées : numpy=42, tf=42, python_random=42.
- Découpage train/val/test : train_test_split stratifié, seed=42.
- Reproductibilité : tf.config.experimental.enable_op_determinism() dans le notebook P1 ; les CSV/NPY produits sont versionnés via git lfs.

D Extraits de code clés

```
1 class MaskedGlobalAveragePooling1D(layers.Layer):
2     def call(self, x, mask):
3         m = tf.cast(mask, x.dtype)[..., None]          # (B, T, 1)
4         s = tf.reduce_sum(x * m, axis=1)                # (B, C)
5         n = tf.reduce_sum(m, axis=1) + 1e-6
6         return s / n
```

Listing 3 – Pooling global moyenné masqué (`mctnet_model.py`).

```
1 class CrossModalSequenceAttention(layers.Layer):
2     def __init__(self, d_model, num_heads=4, **kw):
3         super().__init__(**kw)
4         self.mha = layers.MultiHeadAttention(num_heads=num_heads,
5                                              key_dim=d_model // num_heads)
6         self.ln = layers.LayerNormalization()
7     def call(self, q, kv):
8         h = self.mha(query=self.ln(q), key=k, value=k)
9         return q + h
```

Listing 4 – Attention croisée modale (`partie3/mctnet_v2_model.py`).